
Project στις Δομές Δεδομένων

Περιγραφή Αταξινόμητου Πίνακα:

Ο αταξινόμητος πίνακας είναι η λιγότερο αποδοτική δομή για την αναζήτηση μίας ή περισσότερων λέξεων. Αρχικά, στον αταξινόμητο πίνακα χρησιμοποιούμε σειριακή αναζήτηση για να αναζητήσουμε τις λέξεις του τυχαίου συνόλου Q με αποτέλεσμα η δομή αυτή να εκτελείται πιο αργά από τις υπόλοιπες. Για την δομή Insert στον αταξινόμητο χρησιμοποιούμε και πάλι την μέθοδο της σειριακής αναζήτησης για να ελέγξουμε εάν η λέξη που θα εισάγουμε στον πίνακα βρίσκεται ήδη μέσα σε αυτόν. Εάν βρίσκεται ήδη τότε αυξάνουμε τον μετρητή times κατά 1 της συγκεκριμένης λέξης το οποίο αντιπροσωπεύει τις εμφανίσεις της κάθε λέξης στο κείμενο. Εάν δεν βρίσκεται ήδη η λέξη μέσα στον πίνακα, εισάγουμε την λέξη στο τέλος του πίνακα και αναλόγως αν δεν υπάρχουν άλλες κενές θέσεις, τότε διπλασιάζουμε δυναμικά το μέγεθος του πίνακα. Για την δομή Delete και πάλι χρησιμοποιούμε την μέθοδο της σειριακής αναζήτησης για να ελέγξουμε εάν η λέξη που θα διαγράψουμε από τον πίνακα βρίσκεται ήδη μέσα σε αυτόν. Εάν βρίσκεται ήδη τότε μειώνουμε τον μετρητή times κατά 1. Εάν όχι τότε επιστρέφουμε false διότι δεν βρέθηκε η λέξη για διαγραφή.

Συνολικά η δομή αυτή είναι η πιο χρονοβόρα.

ΣΕΙΡΙΑΚΗ ΑΝΑΖΗΤΗΣΗ : **Worst-case performance: $O(n)$**

Best-case performance: $O(1)$

Average performance: $O(n/2)$

Περιγραφή Ταξινομημένου Πίνακα:

Ο ταξινομημένος πίνακας είναι εννοείται πιο αποδοτικός για την αναζήτηση λέξεων απ' ότι ο αταξινόμητος. Αρχικά εισάγουμε την πρώτη λέξη στην πρώτη θέση του πίνακα, μετά χρησιμοποιούμε την μέθοδο της δυαδικής αναζήτησης(η οποία είναι πολύ πιο γρήγορη από την σειριακή) για να ελέγξουμε εάν βρίσκεται η κάθε λέξη μέσα στον πίνακα ή όχι. Εάν η λέξη βρίσκεται μέσα στον πίνακα τότε αυξάνουμε τον μετρητή times στην συγκεκριμένη θέση κατά 1. Εάν η λέξη δεν υπάρχει μέσα στον πίνακα υπάρχουν 3 περιπτώσεις: α) Η λέξη να είναι μικρότερη από όλες τις λέξεις του πίνακα και τότε την τοποθετούμε στην πρώτη θέση του πίνακα μετακινώντας όλα τα άλλα στοιχεία 1 θέση δεξιά, β) η λέξη να είναι μεγαλύτερη από όλες τις λέξεις κι έτσι καταχωρείται στο τέλος του πίνακα, γ) η λέξη να βρίσκεται ενδιάμεσα και με την βοήθεια της μεθόδου της δυαδικής αναζήτησης βρίσκουμε τις 2 λέξεις που πρέπει να είναι δεξιά και αριστερά από την συγκεκριμένη. Εάν δεν υπάρχουν άλλες κενές θέσεις για εισαγωγή καινούργιου στοιχείου στον πίνακα, τότε διπλασιάζουμε δυναμικά το μέγεθος του. Στην δομή Delete με την βοήθεια της δυαδικής αναζήτησης βρίσκουμε την θέση που θέλουμε να διαγράψουμε και μειώνουμε τον μετρητή times κατά 1. Εάν ο μετρητής μηδενιστεί τότε διαγράφεται όλη εκείνη η θέση του πίνακα.

Συνολικά ο ταξινομημένος είναι πιο αργός στην κατασκευή αλλά πολύ πιο γρήγορος στην αναζήτηση από τον αταξινόμητο.

ΔΥΑΔΙΚΗ ΑΝΑΖΗΤΗΣΗ: **Worst-case performance: $O(\log n)$**

Best-case performance: $O(1)$

Average performance: $O(\log n)$

Περιγραφή Δυαδικού Δέντρου Αναζήτησης :

Τα δυαδικά δέντρα αναζήτησης είναι δομές οι οποίες χρησιμοποιούνται στην αναζήτηση στοιχείων, όπου ενδέχεται να βελτιώσουν σημαντικά το χρόνο. Αρχικά, στην δομή μας εισάγουμε την ρίζα του δέντρου η οποία είναι η πρώτη λέξη του κειμένου και τότε αν η λέξη είναι λεξικογραφικά μεγαλύτερη από την ρίζα ψάχνουμε στο δεξιό μέρος του δέντρου να τοποθετήσουμε την λέξη αυτή. Αργότερα κάνουμε μετάβαση από Node σε Node χρησιμοποιώντας αναδρομή(εάν η λέξη είναι μικρότερη λεξικογραφικά μεταβαίνουμε αριστερά ενώ αν είναι μεγαλύτερη πάμε δεξιά) κάνουμε σύγκριση της λέξης με το Tree Node μέχρι να βρεθεί η ίδια λέξη στο δέντρο για να αυξάνουμε τον μετρητή της συγκεκριμένης λέξης ή αν δεν υπάρχει αυτή η λέξη στο δέντρο δημιουργούμε νέο Tree Node με αυτή τη λέξη και υπολογίζουμε το νέο ύψος του δέντρου ανάλογα. Η ίδια διαδικασία ακολουθείται και στο αριστερό μέρος του δέντρου εάν η λέξη είναι μικρότερη από την ρίζα. Για την διαγραφή χρησιμοποιώντας την πιο πάνω αναδρομική μέθοδο για να αναζητήσουμε τον κόμβο που θέλουμε να διαγράψουμε υπάρχουν 3 περιπτώσεις που μπορούν να προκύψουν : α) Εάν ο κόμβος έχει ένα παιδί τότε μετά τη διαγραφή του κόμβου τη θέση του παίρνει η ρίζα του παιδιού και ενημερώνεται κατάλληλα ο δείκτης του γονιού του, β) Εάν ο κόμβος δεν έχει παιδιά τότε απλά διαγράφουμε τον κόμβο και θέτουμε τον δείκτη του γονιού σε NULL, γ) Εάν ο κόμβος έχει δύο παιδιά τότε τη θέση του στοιχείου που διαγράφεται θα πάρει είτε το μεγαλύτερο στοιχείο του αριστερού παιδιού είτε το μικρότερο στοιχείο του δεξιού παιδιού.

Συνολικά η κατασκευή του δυαδικού αλλά και η αναζήτηση είναι πολύ γρήγορη.

ΔΥΑΔΙΚΟ ΔΕΝΤΡΟ ΑΝΑΖΗΤΗΣΗΣ (Εάν ύψος=h) : **$O(h)$**

(Όσο μικρότερο το
Ύψος τόσο καλύτερες
Αποδόσεις)

Worst-case performance: **$O(\log n)$**

Περιγραφή Ισοζυγισμένου(AVL) Δυαδικού Δέντρου:

Η διαφορά ενός Ισοζυγισμένου(AVL) δυαδικού δέντρου αναζήτησης με ένα κανονικό δυαδικό δέντρο αναζήτησης είναι ότι η διαφορά των υψών του αριστερού και του δεξιού υποδέντρου δεν πρέπει να ξεπερνά το 1 και αυτό πρέπει να ισχύει για όλους τους κόμβους. Η αναζήτηση σε ένα AVL δέντρο είναι ίδια με ενός απλού δυαδικού δέντρου. Όμως μετά από κάθε εισαγωγή ή διαγραφή πρέπει να γίνεται έλεγχος εάν ισχύει η διαφορά υψών μεταξύ κόμβων που αναφέραμε παραπάνω, έτσι αν δεν ισχύει πρέπει να γίνουν δομικές αλλαγές στο δέντρο ώστε να γίνει και πάλι ισοζυγισμένο. Έτσι στο πρόγραμμα μας χρησιμοποιούμε 4 ειδών περιστροφών για την επίτευξη του στόχου μας : α) Απλή δεξιά περιστροφή , β) Απλή αριστερή περιστροφή, γ) Διπλή δεξιά περιστροφή, δ) Διπλή αριστερή περιστροφή.

Συνολικά η κατασκευή του Ισοζυγισμένου(AVL) δυαδικού δέντρου είναι συνήθως πιο χρονοβόρα από ότι ένα απλό δέντρο αλλά η αναζήτηση σ' αυτό είναι συνήθως γρηγορότερη.

Οι χρόνοι είναι παρόμοιοι με το απλό δυαδικό δέντρο.

Περιγραφή Πίνακα Κατακερματισμού:

Ο Πίνακας Κατακερματισμού (Hash Table) είναι μία δομή δεδομένων για την αποθήκευση συνόλων στοιχείων. Χαρακτηριστικό του πίνακα κατακερματισμού είναι ότι μπορεί να εκτελέσει σε σταθερό χρόνο, δηλαδή με πολυπλοκότητα $O(1)$, τις λειτουργίες της εισαγωγής, αναζήτησης και διαγραφής στοιχείων. Σε αντίθεση με τις άλλες δομές, δημιουργούμε από την αρχή ένα πίνακα με πολύ μεγάλο μέγεθος (πολύ μεγάλος πρώτος αριθμός) χωρίς να τον αυξάνουμε δυναμικά. Αρχικά εισάγουμε τις λέξεις του κειμένου μία προς μία μέσα στον πίνακα κατακερματισμού, χρησιμοποιώντας τη συνάρτηση του hash value για να βρούμε την θέση στην οποία θα τοποθετήσουμε την λέξη. Το hash value της κάθε λέξης είναι μοναδικό εκτός στην περίπτωση σύγκρουσης (collision) όπου μία άλλη λέξη βρίσκεται στην θέση που προσπαθούμε να καταχωρίσουμε, κι έτσι αναζητούμε σειριακά το επόμενο κενό για να την εισάγουμε. Αν ψάχνοντας το επόμενο κενό φτάσουμε στο τέλος του πίνακα τότε αρχίζουμε να ψάχνουμε από την $1^{\text{η}}$ (0) θέση του πίνακα. Με την ίδια μέθοδο υλοποιούμε και την αναζήτηση. Ένας ιδανικός πίνακας κατακερματισμού είναι αυτός που δεν έχει συγκρούσεις και άρα δηλαδή κάθε λέξη έχει μοναδικό hash value.

Συνολικά ο πίνακας κατακερματισμού είναι ο πιο γρήγορος αλγόριθμος και στην κατασκευή αλλά και στην αναζήτηση.

ΑΝΑΖΗΤΗΣΗ ΣΤΟΝ ΠΙΝΑΚΑ ΚΑΤΑΚΕΡΜΑΤΙΣΜΟΥ : **$O(1)$**

ΧΡΟΝΟΙ ΔΟΜΩΝ ΣΤΟ ΠΡΟΓΡΑΜΜΑ

```
FILE USED: small-file.txt
All words in file: 251353
Random Number of Words Taken: 100000
It took me 4.2086 seconds to Construct NonSortedArray.
It took me 0.60632 seconds to Search in NonSortedArray.
It took me 10.9269 seconds to Construct SortedArray.
It took me 0.077796 seconds to Search in SortedArray.
It took me 0.119086 seconds to Construct HashTable.
It took me 0.042156 seconds to Search in HashTable.
It took me 0.49166 seconds to Construct BinaryTree.
It took me 0.161568 seconds to Search in BinaryTree.
It took me 0.450904 seconds to Construct AVLTree.
It took me 0.151661 seconds to Search in AVLTree.

Process finished with exit code 0
```

Κατάταξη χρόνων : (Με βάση την πιο πάνω εκτέλεση)

ΚΑΤΑΣΚΕΥΗ	ΑΝΑΖΗΤΗΣΗ
i. Hash Table	i. Hash Table
ii. AVL Binary Tree	ii. Sorted Array
iii. Binary Tree	iii. AVL Binary Tree
iv. Unsorted Array	iv. Binary Tree
v. Sorted Array	v. Unsorted Array

4054 - Αντρέας Κατσονούρης
4023 - Στέφανος Αναστασιάδης